

자료구조 실습 보고서

[제 03 주] 연결 체인 가방

제출일 : 2018 년 4 월 1 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘, 자료구조

이 프로그램은 입출력을 담당하는 AppView 클래스,
전체적인 제어를 담당하는 AppController 클래스,
값을 가지는 동전을 구현하는 Coin 클래스,
연결 체인 구조로 가방을 구현하는 LinkedBag 클래스,
연결 체인을 이루는 노드를 구현하는 Node 클래스로 구성된다.

이번 과제는 2 주차에 배열을 이용하여 Bag 의 구조를 구현하였던 것을 연결 체인을 이용하여 다시 구현해보는 것이다.

연결 체인은 노드들이 연결된 구조를 말하는 것이며, 각각의 노드는 '값' 과 '다음 노드' 의 정보를 담고 있다. 따라서 노드들이 꼬리를 물고 늘어진 체인의 형태를 이루게 된다.

연결 체인은 원소가 추가될 수록 크기가 늘어나기 때문에, 처음에 크기를 지정할 필요가 없으며 인덱스 라는 것이 존재하지 않기 때문에 삽입과 삭제가 배열에 비해 훨씬 효율적이라는 장점을 가진다.

- `public boolean remove(Element givenElement)` : 가방에서 원소 삭제

원소를 제거할 때는 결국 그 원소를 포함하는 노드를 통째로 삭제하게 되는데,
노드를 삭제할 때는 그 노드를 가리키는 참조변수가 없도록 하면 된다. (가비지 컬렉션에 의해 정리됨)

먼저, 함수의 매개변수로 받은 Element 와 같은 값을 가지는 원소를 찾기 위하여 탐색을 해야 한다.
이를 위하여 `currentNode` 라는 임시 노드를 선언하며, 맨 처음에는 맨 앞 노드인 head 를 가리키게 하고 점차적으로 뒤로 이동시킨다.

같은 값을 찾으면 즉시 `break` 문을 통해 탐색을 멈추어 `currentNode` 에 찾고자 하는 노드의 위치가 담기게 되고, 찾지 못하면 연결 체인의 끝까지 탐색하다가 결국 `currentNode` 가 null 이 되어 while 문을 탈출하게 된다.

그 이후 찾은 위치를 기반으로 삭제를 하는데, 찾지 못했을 때에는 바로 false 를 반환시켜준다.
찾은 위치에 따라서 경우가 나뉘게 되며, 맨 앞인 경우와 중간인 경우는 자신을 가리키던 참조변수를 자신 다음을 가리키게 하면 되고, 맨 뒤인 경우에는 그냥 자신을 가리키던 참조변수를 null 로 초기화하면 된다.

그 후 size 를 1 감소시키고 true 를 반환한다.

b. 함수 설명서

■ LinkedBag

- public LinkedBag() : 가방의 현재 크기와, 맨 앞의 노드 초기화 (생성자)
- public int size() : 가방 크기 반환 (Getter)
- public boolean isEmpty() : 가방 비어있는지 여부 반환
- public Element elementAt(int givenOrder) : 인자로 받은 인덱스에 있는 원소 반환 (Getter)
- public boolean doesContain(Element givenElement) : 인자로 받은 원소와 같은 값의 원소가 있는지 여부 반환
- public int frequencyOf(Element givenElement) : 인자로 받은 원소와 같은 값의 원소가 몇 개 있는지 반환
- public boolean add(Element givenElement) : 인자로 받은 원소를 가방에 추가
- public boolean remove(E element_p) : 인자로 받은 원소와 같은 값을 갖는 원소 1 개 제거
- public Element removeAny() : 맨 앞의 원소 삭제한 후, 삭제한 원소 반환
- public void clear() : 가방 비움

■ AppController

- public void run() : 프로그램 실행

■ AppView

- public void outputMessage(String aMessageString) : 문자열 출력
- public void outputResult(int aTestSize, long aTestInsertTime, long aTestFindMaxTime) : 결과 출력 (테스트한 데이터의 크기, 삽입 시 걸린 시간, 최댓값 검색 시간)
- public int inputMenuNumber() : 메뉴 번호 입력받아 반환
- public int inputCoinValue() : 코인 액수 입력받아 반환

■ Coin

- public Coin(int givenValue) : 코인에 값 설정. (생성자)
- public int value() : 코인 값 반환 (Getter)
- public void setValue(int newValue) : 코인 값 설정 (Setter)
- public boolean equals(Object coin_p) : 코인 값 비교 후 같은 지 반환

■ Node

- public Node() : 원소와 다음 노드 위치 정보 초기화 (생성자)

- public Node(Element givenElement) : 원소 직접 설정, 다음 노드 위치 정보 초기화 (생성자)
- public Node(Element givenElement, Node<Element> givenNext) : 원소와 다음 노드 위치 정보 직접 설정 (생성자)
- public Element element() : 현재 노드의 원소 반환 (Getter)
- public Node<Element> next() : 다음 노드 위치 정보 반환 (Getter)
- public void setElement(Element givenElement) : 현재 노드의 원소 설정 (Setter)
- public void setNext(Node<Element> givenNext) : 다음 노드 설정 (Setter)

c. 종합 설명서

이번 과제는 2 주차 과제에서 설계하였던 Unsorted ArrayBag 프로그램에서, Node 와 LinkedBag 만 변경 한 후 나머지는 거의 수정하지 않은 것으로, 크기를 지정할 필요가 없고 remove 의 입력에 음수를 입력할 시 맨 앞의 원소를 삭제한다는 점을 제외하고는 전부 2 주차 과제와 같다.

프로그램을 실행하면 바로 메뉴가 나오며, 동전의 추가/제거/존재여부확인/빈도수확인, 종료를 할 수 있다.

2. 프로그램 장단점 분석

장점 : 연결 체인은 원소가 추가될 수록 크기가 늘어나기 때문에 처음에 크기를 지정할 필요가 없으며, 인덱스 라는 것이 존재하지 않기 때문에 삽입과 삭제시에는 원소들의 인덱스를 옮겨줘야 하는 배열에 비해 훨씬 효율적이라는 장점을 가진다.

단점 : 코인의 값으로 0 도 취급한다.

메뉴 번호를 입력받을 때 정수가 아닌 값을 입력하면 InputMismatchException 이 발생한다.

3. 실행 결과 분석

a. 입력과 출력

<<< 동전 가방 프로그램을 시작합니다. >>>

수행하려는 메뉴 번호 입력
(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 1
코인 액수 입력 : 10
주어진 값의 동전을 가방에 성공적으로 넣었습니다.

수행하려는 메뉴 번호 입력
(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 2
코인 액수 입력 : 10
주어진 값을 갖는 동전 하나가 가방에서 정상적으로 삭제되었습니다.

수행하려는 메뉴 번호 입력
(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 1
코인 액수 입력 : 100
주어진 값의 동전을 가방에 성공적으로 넣었습니다.

수행하려는 메뉴 번호 입력
(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 1
코인 액수 입력 : 500
주어진 값의 동전을 가방에 성공적으로 넣었습니다.

```

수행하려는 메뉴 번호 입력
(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 3
코인 액수 입력 : 100
주어진 값을 갖는 동전이 가방 안에 존재합니다.

수행하려는 메뉴 번호 입력
(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 4
코인 액수 입력 : 500
주어진 값을 갖는 동전의 개수는 1 개 입니다.

수행하려는 메뉴 번호 입력
(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 123123
선택된 메뉴 번호 123123는 잘못된 번호입니다.

수행하려는 메뉴 번호 입력
(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : 9

총 코인의 개수 : 2
가장 큰 코인 금액 : 500
전체 금액 : 600
<<< 동전 가방 프로그램을 종료합니다. >>>

```

b. 결과분석

기본적으로 2 주차 과제에서 가방의 구현 방식만 바꾸고 나머지는 그대로 보존하는 것을 목표로 하였기 때문에, 입력값을 2 주차와 똑같이 해보았다.

그 결과, 같은 결과를 얻을 수 있었다.

먼저 10 코인을 하나 넣었다가 뺐고, 100 코인과 500 코인을 하나씩 넣었다.

3 번(존재 여부 확인) 메뉴에서 100 코인이 존재한다고 하였고,

4 번(빈도수 확인) 메뉴에서 500 코인이 1 개 존재한다고 하였으며

9 번(종료) 의 결과에서 600 원으로 나왔으므로 맞는 결과가 나왔다고 할 수 있다.

4. 소스코드

AppController.java

```

public class AppController {

    // 상수
    private static final int MENU_ADD = 1;
    private static final int MENU_REMOVE = 2;
    private static final int MENU_SEARCH = 3;
    private static final int MENU_FREQUENCY = 4;
    private static final int MENU_EXIT = 9;

    // 인스턴스 변수
    private AppView _appView;
    private LinkedBag<Coin> _coinBag;

```

```

// 생성자
public AppController() {
    this._appView = new AppView();
}

// Coin 추가
private void addCoin() {
    int coinValue = this._appView.inputCoinValue();
    if(this._coinBag.add(new Coin(coinValue))) {
        this._appView.outputLine("주어진 값의 동전을 가방에 성공적으로 넣었습니다.");
    }
    else {
        this._appView.outputLine("주어진 값의 동전을 가방에 넣는데 실패하였습니다.");
    }
}

// Coin 제거
private void removeCoin() {

    // 삭제할 코인 값 입력
    int coinValue = this._appView.inputCoinValue();

    // 음수를 입력받았을 때 : 맨 앞의 코인 제거
    if(coinValue < 0) {
        Coin removedCoin = this._coinBag.removeAny();
        if(removedCoin == null) {
            this._appView.outputLine("가방에 동전이 존재하지 않습니다.");
        }
        else {
            this._appView.outputLine("가방에서 동전 " + removedCoin.value() + " 를
제거하였습니다.");
        }
    }

    // 음수가 아닌 값을 입력받았을 때 : 해당 값의 코인 제거
    else {
        if(!this._coinBag.remove(new Coin(coinValue))) {
            this._appView.outputLine("주어진 값을 갖는 동전은 가방 안에 존재하지
않습니다.");
        }
        else {
            this._appView.outputLine("주어진 값을 갖는 동전 하나가 가방에서
정상적으로 삭제되었습니다.");
        }
    }
}

```

```

// Coin 존재 유무 확인
private void searchForCoin() {
    int coinValue = this._appView.inputCoinValue();
    if(this._coinBag.contains(new Coin(coinValue))) {
        this._appView.outputLine("주어진 값을 갖는 동전이 가방 안에 존재합니다.");
    }
    else {
        this._appView.outputLine("주어진 값을 갖는 동전은 가방 안에 존재하지
않습니다.");
    }
}

// Coin 개수 출력
private void frequencyOfCoin() {
    int coinValue = this._appView.inputCoinValue();
    int frequency = this._coinBag.frequencyOf(new Coin(coinValue));
    this._appView.outputLine("주어진 값을 갖는 동전의 개수는 " + frequency + " 개 입니다.");
}

// 번호 잘못 입력 시 오류 출력
private void undefinedMenuNumber(int menuNumber_p) {
    this._appView.outputLine("선택된 메뉴 번호 " + menuNumber_p + "는 잘못된
번호입니다.");
}

// 전체 Coin의 값 합계 반환
private int sumOfCoinValues() {
    int sum = 0;
    for(int index = 0; index < this._coinBag.size(); index++) {
        sum += this._coinBag.elementAt(index).value();
    }
    return sum;
}

// 가장 큰 Coin의 값 반환
private int maxCoinValue() {
    int maxCoinValue = 0;
    for(int index = 0; index < this._coinBag.size(); index++) {
        if(maxCoinValue < this._coinBag.elementAt(index).value()) {
            maxCoinValue = this._coinBag.elementAt(index).value();
        }
    }
    return maxCoinValue;
}

// 가방 내용 정보 출력
private void showStatistics() {

```

```

        this._appView.outputLine("");
        this._appView.outputLine("총 코인의 개수 : " + this._coinBag.size());
        this._appView.outputLine("가장 큰 코인 금액 : " + this.maxCoinValue());
        this._appView.outputLine("전체 금액 : " + this.sumOfCoinValues());
    }

```

// 시작

```

public void run() {

    // 시작 메시지
    this._appView.outputLine("<<< 동전 가방 프로그램을 시작합니다. >>>");

    // 가방에 넣을 동전 개수 입력 : 필요 x
    // int capacityOfCoinBag = this._appView.inputCapacityOfCoinBag();
    this._coinBag = new LinkedBag<Coin>();

    // 메뉴 번호 입력
    int menuNumber = this._appView.inputMenuNumber();
    while(menuNumber != MENU_EXIT) {

        switch(menuNumber) {

            case MENU_ADD:
                this.addCoin();
                break;

            case MENU_REMOVE:
                this.removeCoin();
                break;

            case MENU_SEARCH:
                this.searchForCoin();
                break;

            case MENU_FREQUENCY:
                this.frequencyOfCoin();
                break;

            default:
                this.undefinedMenuNumber(menuNumber);
        }

        // 새로운 메뉴 번호 입력
        menuNumber = this._appView.inputMenuNumber();
    }

    // 가방 안의 동전 정보 출력
    this.showStatistics();

    // 종료 메시지

```



```
        this._appView.outputLine("<<< 동전 가방 프로그램을 종료합니다. >>>");  
    }  
}
```

AppView.java

// 입출력

```
import java.util.Scanner;
```

```
public class AppView {
```

// 인스턴스 변수

```
private Scanner _scanner;
```

// 생성자

```
public AppView() {  
    this._scanner = new Scanner(System.in);  
}
```

// 입력 메소드

/*

// 코인 개수 입력받아 반환

```
public int inputInt() {
```

```
    int coinAmount;
```

```
    this.outputLine("");
```

```
    this.output("가방에 들어갈 총 코인 개수 입력 : ");
```

```
    coinAmount = _scanner.nextInt();
```

```
    return coinAmount;
```

```
}
```

*/

// 메뉴 번호 입력받아 반환

```
public int inputMenuNumber() {
```

```
    int menuNumber;
```

```
    this.outputLine("");
```

```
    this.outputLine("수행하려는 메뉴 번호 입력");
```

```
    this.output("(add : 1, remove : 2, search : 3, frequency : 4, exit : 9) : ");
```

```
    menuNumber = this._scanner.nextInt();
```

```
    return menuNumber;
```

```
}
```

// 코인 액수 입력받아 반환

```
public int inputCoinValue() {
```

```

        int coinValue;
        this.output("코인 액수 입력 : ");
        coinValue = this._scanner.nextInt();

        return coinValue;
    }

    /* 가방에 들어갈 최대 코인 개수 입력받아 반환 : 필요 x
    public int inputCapacityOfCoinBag() {

        int capacityOfCoinBag;
        this.output("가방에 들어갈 코인의 최대 개수 입력 : ");
        capacityOfCoinBag = _scanner.nextInt();

        return capacityOfCoinBag;
    }*/

    // 출력 메소드

    // 출력
    public void output(String string_p) {
        System.out.print(string_p);
    }

    // 줄바꿈 출력
    public void outputLine(String string_p) {
        System.out.println(string_p);
    }
}

```

LinkedBag.java

// 가방에 대한 정보

```

public class LinkedBag<Element> {

    private int _size;                // 현재 가방이 가지고 있는 원소의 개수
    private Node<Element> _head;      // 맨 앞 노드

    // 생성자
    public LinkedBag() {
        this._size = 0;
        this._head = null;
    }
}

```

// 현재 크기 반환 : getter

```
public int size() {  
    return this._size;  
}
```

// 가방 비어있는지 여부 반환

```
public boolean isEmpty() {  
    return (this._size == 0 && this._head == null);  
}
```

/* 가방 다 찼는지 여부 반환 : 필요 x

```
public boolean isFull() {  
    return false;  
}*/
```

// 주어진 order에 있는 원소 반환

```
public Element elementAt(int givenOrder) {  
  
    Node<Element> currentNode = this._head;  
  
    // order만큼 뒤에 있는 원소의 정보를 currentNode에 저장  
    for(int index = 0; index < givenOrder; index++) {  
        currentNode = currentNode.next();  
    }  
  
    return currentNode.element();  
}
```

// 주어진 원소와 같은 값의 원소가 있는지 여부 반환

```
public boolean contains(Element givenElement) {
```

```
    boolean found = false;
```

```
    Node<Element> currentNode = this._head;
```

// 탐색을 위한 임시 노드, head부터

시작

// 아직 찾지 못했고, 현재 노드가 존재할 때

```
while(!found && (currentNode != null)) {  
    if(currentNode.element().equals(givenElement))  
        found = true;  
    currentNode = currentNode.next();  
}
```

```
return found;
```

```
}
```

// 주어진 원소와 같은 값의 원소가 몇 개 있는지 반환

public int frequencyOf(Element givenElement) {

int count = 0;

Node<Element> currentNode = **this**._head;

// 탐색을 위한 임시 노드, head부터

시작

// 현재 노드가 존재하고, 원소 또한 존재할 때

while(currentNode != **null** && currentNode.element() != **null**) {

if(currentNode.element().equals(givenElement))

count++;

currentNode = currentNode.next();

}

return count;

}

// 가방에 원소 추가

public boolean add(Element givenElement) {

Node<Element> nodeToAdd = **new** Node<Element>(givenElement, **this**._head);

this._head = nodeToAdd;

this._size++;

return true;

}

// 가방에서 주어진 값의 원소 하나 삭제 후 양옆의 노드 연결

public boolean remove(Element givenElement) {

// 비어 있을 때

if(**this**.isEmpty())

return false;

// 노드가 존재

else {

boolean found = **false**;

Node<Element> previousNode = **null**;

Node<Element> currentNode = **this**._head;

// 탐색을 위한 임시 노드, head부터

시작

// 1. 탐색

// 아직 찾지 못했고, 현재 노드가 존재할 때

while(!found && (currentNode != **null**)) {

if(currentNode.element().equals(givenElement)) {

```

        found = true;
        break;          // 찾으면 이동 중지
    }

    previousNode = currentNode;
    currentNode = currentNode.next();

}

// 2. 삭제

// 찾지 못했을 때
if(found == false) {
    return false;
}

// 찾은 위치가 처음일 때 삭제
else if(currentNode == this._head)
    this._head = this._head.next();

// 찾은 위치가 끝일 때 삭제
else if(currentNode.next() == null)
    previousNode.setNext(null);

// 찾은 위치가 중간일 때 삭제
else {
    previousNode.setNext(currentNode.next());
}

this._size--;

return true;
}

}

// 맨 앞 원소 삭제 후, 삭제한 원소 반환
public Element removeAny() {
    if(this.isEmpty()) {
        return null;
    }
    else {
        Element removedElement = this._head.element();
        this._head = this._head.next();
        this._size--;
        return removedElement;
    }
}

// 가방 초기화

```

```
        public void clear() {
            this._size = 0;
            this._head = null;
        }
    }
}
```

Coin.java

// 동전에 대한 정보

```
public class Coin {

    // 인스턴스 변수

    // 금액
    private int _value;

    // 생성자
    public Coin() { }
    public Coin(int givenValue) {
        this._value = givenValue;
    }

    // 코인 금액 반환 : getter
    public int value() {
        return this._value;
    }

    // 코인 금액 설정 : setter
    public void setValue(int newValue) {
        this._value = newValue;
    }

    // 코인 금액 비교
    @Override
    public boolean equals(Object coin_p) {

        // 인자로 받은 값이 Coin형이 아닐 경우
        if(coin_p.getClass() != Coin.class) {
            return false;
        }

        // Coin형을 받았을 경우 금액 비교
        else {
            return (this.value() == ((Coin)coin_p).value());
        }
    }
}
```

```
}
```

Node.java

```
public class Node<Element> {

    private Element _element;    // 현재 노드의 원소
    private Node<Element> _next;    // 다음 노드

    // 생성자

    // 디폴트
    public Node() {
        this._element = null;
        this._next = null;
    }

    // 원소만 제공
    public Node(Element givenElement) {
        this._element = givenElement;
        this._next = null;
    }

    // 원소와 다음 노드 제공
    public Node(Element givenElement, Node<Element> givenNext) {
        this._element = givenElement;
        this._next = givenNext;
    }

    // Getter
    public Element element() {
        return this._element;
    }
    public Node<Element> next() {
        return this._next;
    }

    // Setter
    public void setElement(Element givenElement) {
        this._element = givenElement;
    }
    public void setNext(Node<Element> givenNext) {
        this._next = givenNext;
    }

}
```

DS1_03_201702081_최재범

// main

```
public class DS1_03_201702081_최재범 {  
  
    public static void main(String[] args) {  
        AppController appController = new AppController();  
        appController.run();  
    }  
  
}
```